

Data Science for Innovation

W2: Data Acquisition and Exploration with Python

Presented by
Dr Ali Anaissi

Assignment 1: Project Stage 1



Project stage 1: Data Exploration and Cleaning

Objective

Explore a data set and define a research question based on research/business requirement.

Activities

- Choose a data set
- Explore and summarise data set
- Clean and prepare data
- Define problem
- Do a literature review about the problem

Output

- **3-page report** summarising data, literature review, problem and explorative analysis
 - how did you acquire the data?
 - which tools did you use to clean and explore the data set?
 - with title page & references: max 4p

Marking

- 20% of overall mark

Marking Criteria

- Problem Definition and Literature review
- Acquisition of a relevant Data Set
- Data Preparation (Cleaning & Transformation)
- Data Summarization (Explorative Analysis)
- Report Structure and Style
- Appropriateness of Tools

Detailed marking rubric will be published on Canvas soon!

Suggested timeline for Assignment 1 (Project Stage 1)

- W1: Identify possible data sets
- **W2: Identify and Explore possible data sets**
- *W3: Select project data set, define problem, complete exploration*
- W4: Draft summary (problem & exploratory analysis)
- W5: Clean and prepare data, descriptive Stats, justification of suitability

Types of projects to consider

- Discover clusters in data
- Learn association rules
- Train a classifier and evaluate prediction accuracy
- Train a regression model and evaluate prediction accuracy

Overview of Week 2

Today: Data Exploration with Python

Objective

Learn Python tools for exploring a new data set programmatically.

Lecture

- Data types, cleaning, preprocessing
- Descriptive statistics, e.g., median, quartiles, IQR, outliers
- Descriptive visualisation, e.g., boxplots, confidence intervals

Readings

- [Data Science from Scratch](#): Ch 4-5

Exercises

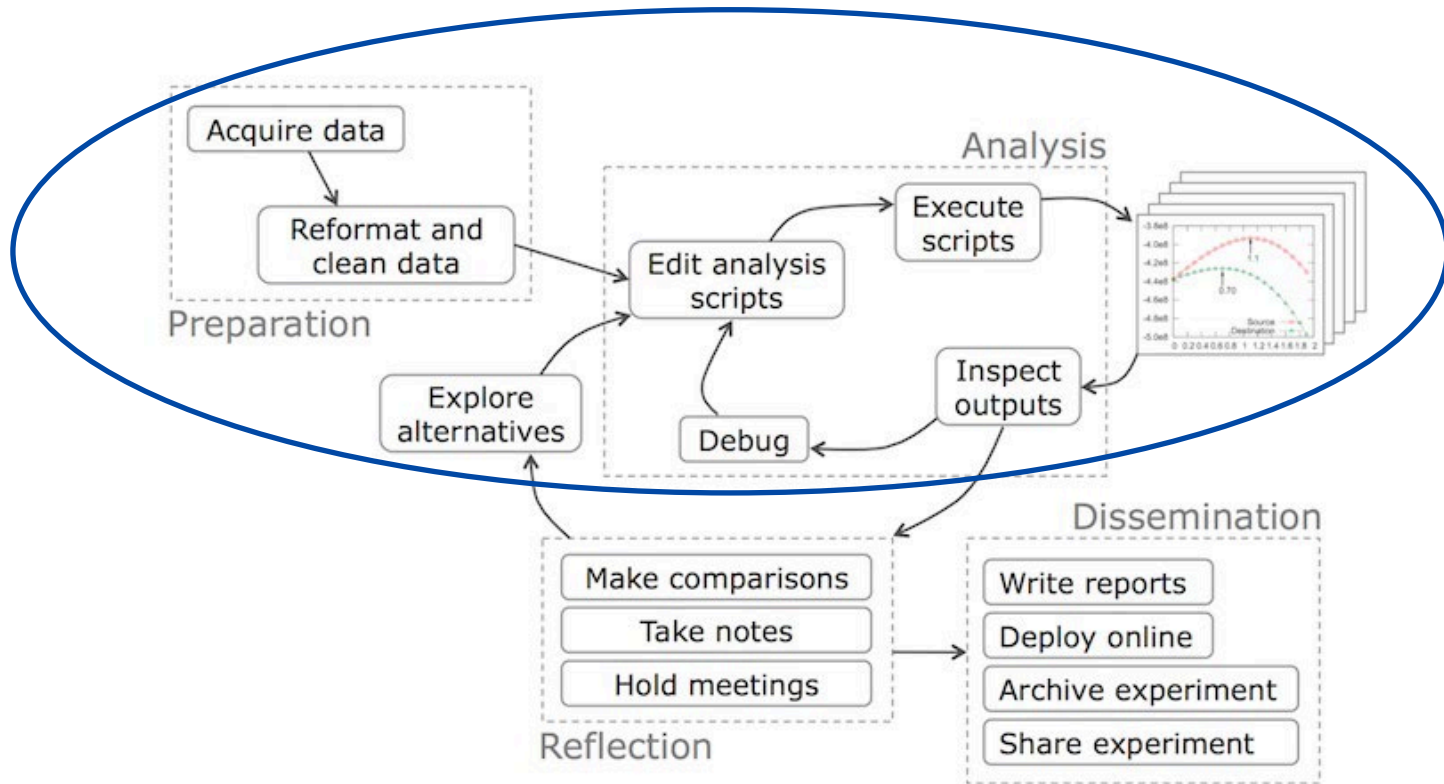
- matplotlib: Visualisation
- numpy/scipy: Descriptive stats

TODO in W2

- Learning Python
- Explore and select project data



Exploratory Analysis Workflow





Preliminaries: Types of Data



Nominal Data

What main industry have you worked in? *

Choose



What key experience do you have? *

- ☐ Relational databases
- ☐ NoSQL
- ☐ Information retrieval

- Values are names
- No ordering is implied
- Eg jersey numbers



Ordinal Data

How important are the following?

Data management *

	1	2	3	4	5	
Not important in the slightest	☐	☐	☐	☐	☐	Extremely important

Statistics *

	1	2	3	4	5	
Not important in the slightest	☐	☐	☐	☐	☐	Extremely important

- Values are ordered
- No distance is implied
- Eg rank, agreement
- *central tendency* can be measured by mode¹ or median
- the mean cannot be defined from an ordinal set
- dispersion can be estimated by the Inter-Quartile Range (IQR)

¹The mode is the number that is repeated more often than any other



- [illegible]

- [1,1,2,2,2,2,2,2,2,2,2,3,3,3, 3] [3,3,3,3,3,3,3,3,3,3,3,3,3, 3]
[3,3,3,3,3,4,4,4,4,4,4,4,4, 4] [4,4,4,4,4,4,4,4,5,5,5,5,5, 5]

the 'cut-off' points
are called **quartiles**

- The IQR is the difference between the first and third quartile. i.e:
 $Q3 - Q1 = 4 - 3 = 1.$



Interval Data



“[Thermometer](#)” by [Christer Edvartsen](#) is licenced under [CC BY 2.0](#)

- Interval scales provide information about order, and also possess equal intervals
- Values encode differences
- equal intervals between values
- No true zero
- Addition is defined
- Eg Celsius temperature
- *central tendency* can be measured by mode, median, or mean



Ratio Data

How many years professional experience do you have? *

Your answer

How many years programming experience do you have? *

Your answer

- Values encode differences
- Zero is defined
- Multiplication defined
- Ratio is meaningful
- Eg length, weight, income



Levels of Measurement

	Nominal	Ordinal	Interval	Ratio
Countable	✓	✓	✓	✓
Order defined		✓	✓	✓
Difference defined (addition, subtraction)			✓	✓
Zero defined (multiplication, division)				✓



What about text data?

How would you define data science in one sentence? *

Your answer

- Not defined as traditional data type in statistics
- Requires interpretation, coding or conversion
- More in future lectures...

Data Acquisition and Cleaning

Data Acquisition – Where does data come from?

- File Access
 - You or your organisation might already have a data set, or a colleagues provides you access to data.
 - Or: Web Download from an online data server
 - Typical exchange formats: CSV, Excel, sometimes also XML
- Programmatically
 - Scrapping the web (HTML)
 - or using APIs of Web Services (XML/JSON)
 - > Cf. textbook, Ch 9
- Database Access
- Collect data yourself, eg. via a survey

This week: Using data from our online survey from Week 1



Cleaning and Transforming Data

- Real data is often '*dirty*'
- Important to do some data cleaning and transforming first
- Typical steps involved:
 - type and name conversion
 - filtering of missing or inconsistent data
 - unifying semantic data representations
 - matching of entries from different sources
- Later also:
 - **Rescaling** and optional dimensionality reduction



Exercise: What questions can we ask?

- In your group, discuss:
 - Review survey questions from W1: <https://bit.ly/3w6FQtC>
 - List 3 questions we can ask
 - Discuss how you would answer each question with this data



Some descriptive questions

- What areas of data science are considered important?
- How do professional/programming experience compare?
- How does programming experience differ across industries?
- What skills do we know? What would we like to learn?
- Which industries are most desirable? Do past/future differ?
- What skills co-occur most? How strong is the association?
- Is there a significant dependence between experience?
- What terms/topics characterise our DS definitions?

Python and Jupyter Notebooks



Python is great for prototyping

- **Interpreted:** direct execution without compilation
- **Dynamically-typed:** don't have to declare a static type
- **Readable:** easy-to-understand syntax
- **Deployable:** easy to incorporate in applications

Python Recap

- general program syntax
- variables and types
 - integer and float numbers, string types, type conversion
 - list of values (list, array)
- condition statements (if/elif/else)
- for loops, ranges
- functions
 - input(), print(), len(), lower(), upper(), ...
 - nesting of functions; example: `print( len( str.upper() ) )`

Python Import System

- Python provides additional functionality available via **import** statement
 - gives access to classes and functions from various 3rd party modules
 - Example: **csv**: comma-separated file format support

```
import csv
for row in csv.reader( ['one,apple,green', 'two,tomato,red'] ):
    print(row[1])
```

- alternative usages to introduce shortcuts or import only certain functions:

```
import csv as X
from csv import DictReader
```



Python has excellent open-source data libraries

- **scipy**: libraries for scientific and technical computing
- **numpy**: support for large multidimensional arrays and matrices
- **matplotlib**: port of matlab plotting functionality
- **scikit-learn**: machine learning library
- **nltk**: natural language toolkit



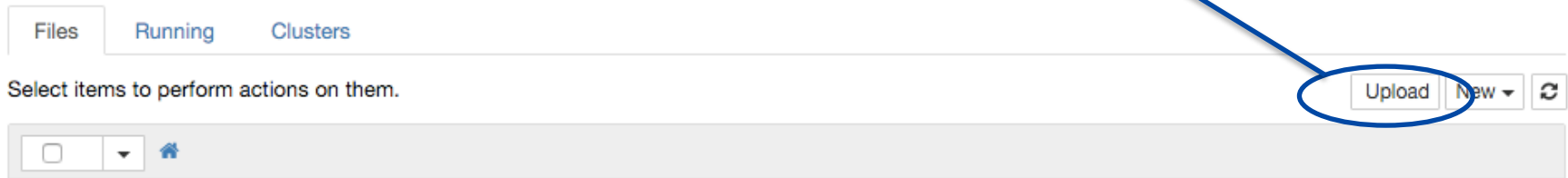
- **pandas**: R-like data frame and associated manipulations



Exercise: Upload survey data and notebook to Jupyter

1. Download data and notebook from Canvas
2. Open your Jupyter notebook
3. Upload data and notebook to your Jupyter

1. Click here for file open dialogue
2. Click upload next to file name





Markdown cell for formatted text

Jupyter notebook cells

Data Exploration with Python

EXERCISE 1: Reading and accessing data

Read the survey response data

The `csv` module supports reading and writing of files in comma-separated values (CSV) and similar formats. We use `DictReader` since the first row of our survey responses file is a header. This produces a list of dictionaries, one dictionary per each individual survey response.

A *dictionary* is a data structure in Python that can hold key-value pairs, where we can lookup values by their key (typically a string, cf. Grok module 9).

The `pprint` command below prints the dictionary corresponding to the first response.

```
import csv
import pprint
data = list(csv.DictReader(open('36100 Survey analysis.csv', encoding="utf-8")))
pprint.pprint(data[0])
```

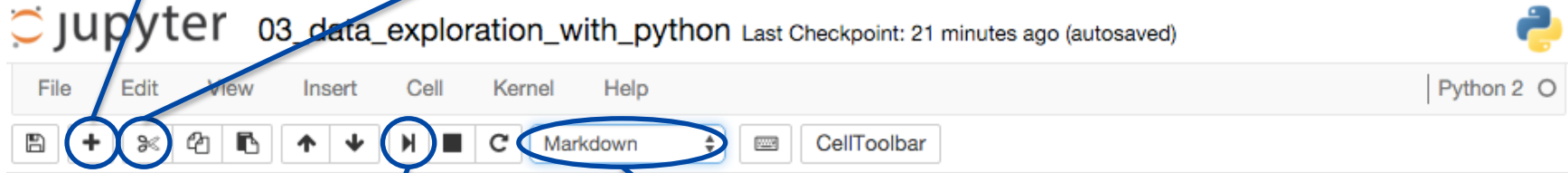
Code cell for writing Python commands



Jupyter menu bar

Add new cell

Cut current cell



“Play” cell

- Execute code
- Render text

Change cell type

Read data using csv

- Python **csv** module
 - Reads/writes comma-separated values with escaping
 - `csv.reader` reads rows into arrays
 - `csv.DictReader` reads rows into ***dictionaries***
- Python **pprint** module
 - pretty print of complex data structures
 - pprint formats a dictionary read by CSV so it's easier to read

```
import csv
import pprint
data = list(csv.DictReader(open('36100 Survey analysis.csv',encoding="utf-8")))
pprint.pprint(data[0])
```

EXERCISE 1: Read survey data into notebook

- Execute read cells
 -  code cell after “Read the survey response data”
 -  code cell after “Define constants for dictionary keys”
- Access data
 - What Python data types do we get from the csv reader?
 - What is the third respondent’s rating for communication?
- **Tip:** All cells where your input is needed are marked **TODO**

```
# TODO: replace the content of this cell with your Python solution  
raise NotImplementedError
```

 - For participation marking, we will check the content of these TODO cells
=> replace the content of those cells with your own solution

Descriptive Statistics

Frequency distributions using `collections.Counter`

- The **`collections`** module provides several useful data structures
- **`Counter`**
 - Takes a list or other iterable as input and counts its entries
 - Returns a count of how often each item appears

```
from collections import Counter
```

```
counts = Counter()
```

```
for row in data:  
    counts[row[IMPORT_COMMUNICATION]] += 1
```

Loop through all data
and count communication ratings

```
print("Distribution of communication importance ratings:")
```

```
for k, v in sorted(counts.items()):  
    print('{}: {}'.format(k, v))
```

Sort and print each k (value)
v (count) pair

Calculating the mode with collections.Counter

- Recall that the **mode** is the most frequent value
- We can use the `most_common()` method to calculate it

Define a mode function

First argument is the data set

Second argument is the column key

```
def mode(data, column_key):  
    mode_counter = Counter()  
    for row in data:  
        mode_counter[row[column_key]] += 1  
    return mode_counter.most_common(1)[0][0]  
  
# example on how to use the 'mode' function  
print("Communication mode:", mode(data, IMPORT_COMMUNICATION))
```

Calculate the n=1 most common
Values, access the first value,
Return the value (not the count)

EXERCISE 2: Frequency distribution and mode

- Calculate frequency distribution using Counter
 -  code cell under “Counting data”
 - Complete code cell to calculate distributions of industries
- Calculate mode using Counter
 -  code cell under “Calculating the mode”
 - Complete code cell to calculate the mode of industries

Cleaning data: convert to correct types

- The Python csv module reads everything as string types
- Need to convert as appropriate (e.g., int, float, timestamp)
 - `int()` creates integer objects, e.g., -1, 101
 - `float()` creates floating point object, e.g., 3.14, 2.71
 - `datetime.strptime()` creates datetime objects from strings

A function to convert values in a given column

Use “not a number” (*nan*) as default value
numpy knows to ignore for some stats

```
import numpy as np
DEFAULT_VALUE = np.nan

def clean(data, column_key, convert_function, default_value):
    for row in data:
        old_value = row[column_key]
        new_value = default_value
        try:
            new_value = convert_function(old_value)
        except (ValueError, TypeError):
            print('Replacing {} with {} in column {}'.format(row[column_key], new_value, column_key))
            row[column_key] = new_value

clean(data, BACKGROUND_YEARS_PROFESSIONAL, float, DEFAULT_VALUE)
clean(data, BACKGROUND_YEARS_PROGRAMMING, float, DEFAULT_VALUE)
```

A function to convert values in a given column

Define *clean* function that cleans given data

```
import numpy as np
DEFAULT_VALUE = np.nan

def clean(data, column_key, convert_function, default_value):
    for row in data:
        old_value = row[column_key]
        new_value = default_value
        try:
            new_value = convert_function(old_value)
        except (ValueError, TypeError):
            print('Replacing {} with {} in column {}'.format(row[column_key], new_value, column_key))
            row[column_key] = new_value

clean(data, BACKGROUND_YEARS_PROFESSIONAL, float, DEFAULT_VALUE)
clean(data, BACKGROUND_YEARS_PROGRAMMING, float, DEFAULT_VALUE)
```

A function to convert values in a given column

Get original value from row
Set new value to default

```
import numpy as np
DEFAULT_VALUE = np.nan

def clean(data, column_key, convert_function, default_value):
    for row in data:
        old_value = row[column_key]
        new_value = default_value
        try:
            new_value = convert_function(old_value)
        except (ValueError, TypeError):
            print('Replacing {} with {} in column {}'.format(row[column_key], new_value, column_key))
            row[column_key] = new_value

clean(data, BACKGROUND_YEARS_PROFESSIONAL, float, DEFAULT_VALUE)
clean(data, BACKGROUND_YEARS_PROGRAMMING, float, DEFAULT_VALUE)
```

Attempt conversion
catching errors

A function to convert/clean values in a given column

list of known strings
and their numerical equivalent

```
import numpy as np
DEFAULT_VALUE = np.nan

def clean(data, column_key, convert function, default value):
    special_values= {'1 year' : 1.0, '2years' : 2.0, '2 years': 2.0, 'Ten' : 10, 'Half a year': 0.5, '6 months': 0.5}
    for row in data:
        old_value = row[column_key]
        new_value = default_value
        try:
            if old_value in special_values.keys():
                new_value = special_values[old_value]
            else:
                new_value = convert_function(old_value)
        except (ValueError, TypeError):
            print('Replacing {} with {} in column {}'.format(row[column_key], new_value, column_key))
            row[column_key] = new_value

clean(data, BACKGROUND_YEARS_PROFESSIONAL, float, DEFAULT_VALUE)
clean(data, BACKGROUND_YEARS_PROGRAMMING, float, DEFAULT_VALUE)
```

replace known strings
with valid number

Cleaning float data

```
import numpy as np
DEFAULT_VALUE = np.nan

def clean(data, column_key, convert_function, default_value):
    special_values= {'1 year' : 1.0, '2years' : 2.0, '2 years': 2.0, 'Ten' : 10, 'Half a year': 0.5, '6 months': 0.5}
    for row in data:
        old_value = row[column_key]
        new_value = default_value
        try:
            if old_value in special_values.keys():
                new_value = special_values[old_value]
            else:
                new_value = convert_function(old_value)
        except (ValueError, TypeError):
            print('Replacing {} with {} in column {}'.format(row[column_key], new_value, column_key))
        row[column_key] = new_value

clean(data, BACKGROUND_YEARS_PROFESSIONAL, float, DEFAULT_VALUE)
clean(data, BACKGROUND_YEARS_PROGRAMMING, float, DEFAULT_VALUE)
```

Call for professional and programming experience columns

Cleaning date data

Format string for US timestamp
from Google Sheets, e.g.,
“3/1/2016 15:59:56”

```
from datetime import datetime
FMT = "%m/%d/%Y %H:%M:%S"
def str_to_time(s):
    if isinstance(s, datetime):
        return s
    return datetime.strptime(s, FMT)
clean(data, TIMESTAMP, str_to_time, DEFAULT_VALUE)
```

Return a datetime object for string s

Call for timestamp column

Central tendency and dispersion with numpy

- **Numpy** provides various statistics for numeric data
- Median, percentiles, mean, standard deviation, etc
- nan* versions calculate same statistics, ignoring NaN values
- Reference page for numpy statistics:
<http://docs.scipy.org/doc/numpy/reference/routines.statistics.html>

Calculating central tendency and dispersion

Calculate stats for professional and programming experience

```
import numpy as np
for column_key in [BACKGROUND_YEARS_PROFESSIONAL, BACKGROUND_YEARS_PROGRAMMING]:
    v = [row[column_key] for row in data] # grab values
    print(column_key.upper())
    print("* Min..Max: {}".format(np.nanmin(v), np.nanmax(v)))
    print("* Range: {}".format(np.nanmax(v)-np.nanmin(v)))
    print("* Mean: {}".format(np.nanmean(v)))
    print("* Standard deviation: {}".format(np.nanstd(v)))
    print("* Median: {}".format(np.nanmedian(v)))
    q1 = np.nanpercentile(v, 25)
    print("* 25th percentile (Q1): {}".format(q1))
    q3 = np.nanpercentile(v, 75)
    print("* 75th percentile (Q3): {}".format(q3))
    iqr = q3-q1
    print("* IQR: {}".format(iqr))
```

`v = []`

for row in data:

`v.append(row[column_key])`

Calculate min/max, range, mean, standard deviation, median, 25/75th percentiles, inter-quartile range

Binning and Histograms

Create histogram of 7 bins
ranging from 0 to 35

```
v = []
for row in data:
    v.append(row[BACKGROUND_YEARS_PROFESSIONAL])

freqs, bins = np.histogram(v, bins=7, range=(0,35)) # calculate frequencies and bin start/end
for i, freq in enumerate(freqs):
    # Note that bins[i] <= bin_values < bins[i+1]
    bin_str = '[{}..{}]'.format(int(bins[i]), int(bins[i+1]))
    print(bin_str, ': ', freq)
```

EXERCISE 3: Cleaning, Statistics and Histograms

- Clean data
 -  code cell under “Cleaning float data”
 -  code cell under “Cleaning timestamp data”
- Statistics and histograms
 -  code cell under “Statistics with numpy”
 -  code cell under “Binning and histograms”
- More histograms
 - Complete code to calculate histogram of programming experience
 - Complete code to calculate histogram with bin size of 2 (EXTRA)

Visualization

Visualising data with matplotlib

- Matplotlib provides functionality for creating various plots
- Bar charts, line charts, scatter plots, etc
- Reference page for pyplot:
https://matplotlib.org/2.0.2/api/pyplot_api.html
- Documentation:
<https://matplotlib.org/stable/users/index.html>

Creating a bar chart

Use Counter to get frequency distribution
Reorder according to IMPORT_KEYS
Add default count of 0 for missing values

```
from collections import OrderedDict
IMPORT_KEYS = ['1', '2', '3', '4', '5']
def make_importance_plot(data, column_key, title):
    c = Counter(row[column_key] for row in data)
    d = OrderedDict([(k,c[k]) if k in c else (k,0) for k in IMPORT_KEYS])
    # bars are by default width 0.8, so we'll add 0.1 to the left coordinates
    xs = [i+0.1 for i,_ in enumerate(IMPORT_KEYS)]
    plt.bar(xs, d.values())
    plt.ylabel('Number of responses')
    plt.axis([0,5,0,35])
    plt.title(title)
    plt.xticks([i + 0.5 for i, _ in enumerate(IMPORT_KEYS)], IMPORT_KEYS)
    plt.show()
for a in IMPORT_AREAS:
    title = 'Importance of {}'.format(a.lower())
    make_importance_plot(data, a, title)
```

Creating a bar chart

List of indices for x axis

```
from collections import OrderedDict
IMPORT_KEYS = ['1', '2', '3', '4', '5']
def make_importance_plot(data, column_key, title):
    c = Counter(row[column_key] for row in data)
    d = OrderedDict([(k,c[k]) if k in c else (k,0) for k in IMPORT_KEYS])
    # bars are by default width 0.8, so we'll add 0.1 to the left coordinates
    xs = [i+0.1 for i, _ in enumerate(IMPORT_KEYS)]
    plt.bar(xs, d.values())
    plt.ylabel('Number of responses')
    plt.axis([0,5,0,35])
    plt.title(title)
    plt.xticks([i + 0.5 for i, _ in enumerate(IMPORT_KEYS)], IMPORT_KEYS)
    plt.show()
for a in IMPORT_AREAS:
    title = 'Importance of {}'.format(a.lower())
    make_importance_plot(data, a, title)
```

Create bar chart

Set axis ranges:
x: 0..5; y: 0..35

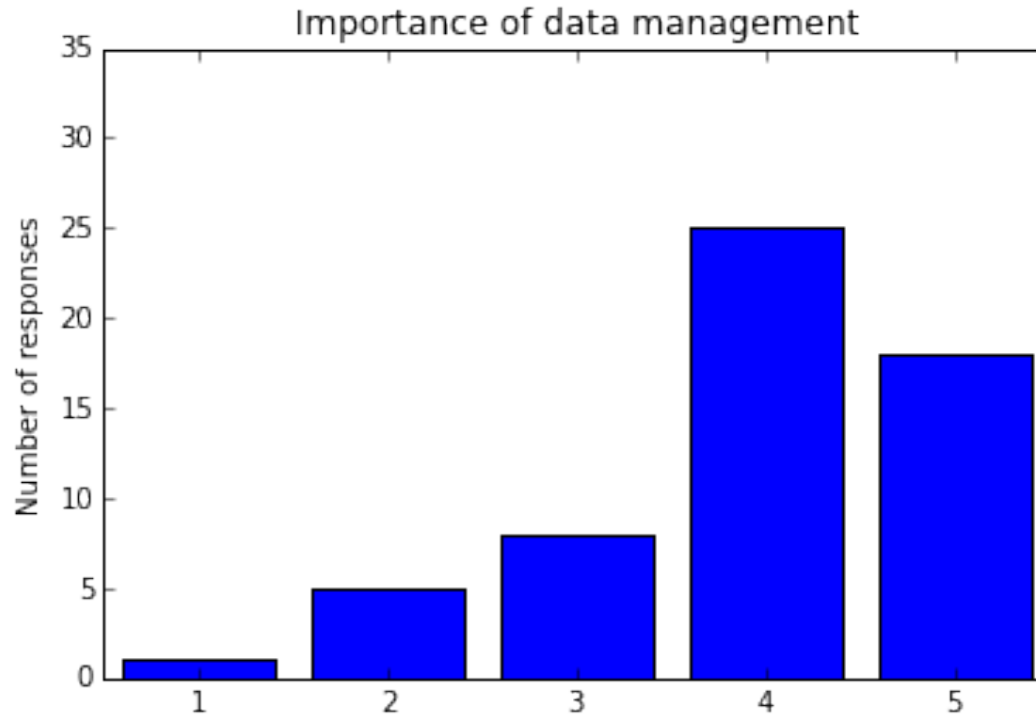
Creating a bar chart

```
from collections import OrderedDict
IMPORT_KEYS = ['1', '2', '3', '4', '5']
def make_importance_plot(data, column_key, title):
    c = Counter(row[column_key] for row in data)
    d = OrderedDict([(k,c[k]) if k in c else (k,0) for k in IMPORT_KEYS])
    # bars are by default width 0.8, so we'll add 0.1 to the left coordinates
    xs = [i+0.1 for i,_ in enumerate(IMPORT_KEYS)]
    plt.bar(xs, d.values())
    plt.ylabel('Number of responses')
    plt.axis([0,5,0,35])
    plt.title(title)
    plt.xticks([i + 0.5 for i, _ in enumerate(IMPORT_KEYS)], IMPORT_KEYS)
    plt.show()
for a in IMPORT_AREAS:
    title = 'Importance of {}'.format(a.lower())
    make_importance_plot(data, a, title)
```

Center x ticks
under bars, and
pass in labels

Make a plot
for each area

A bar chart for data management ratings



Plotting a histogram

Yield bin_label, frequency pairs

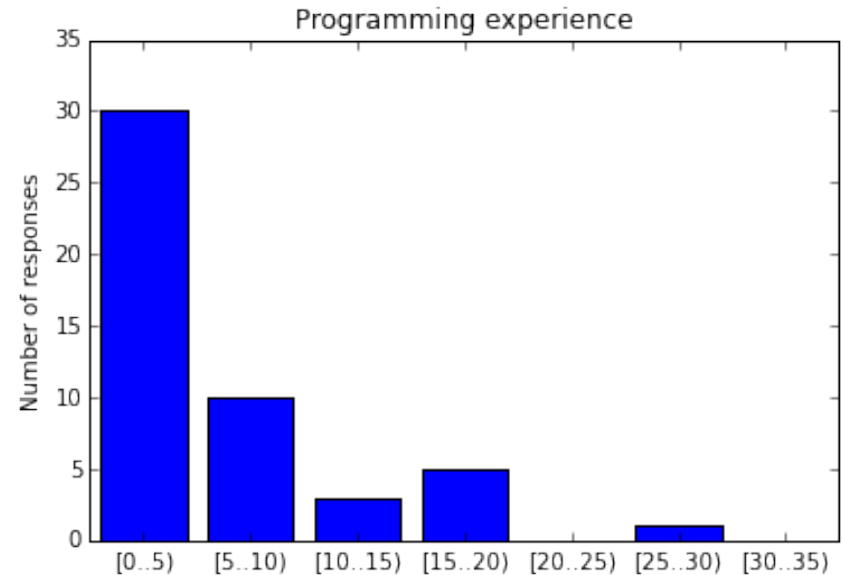
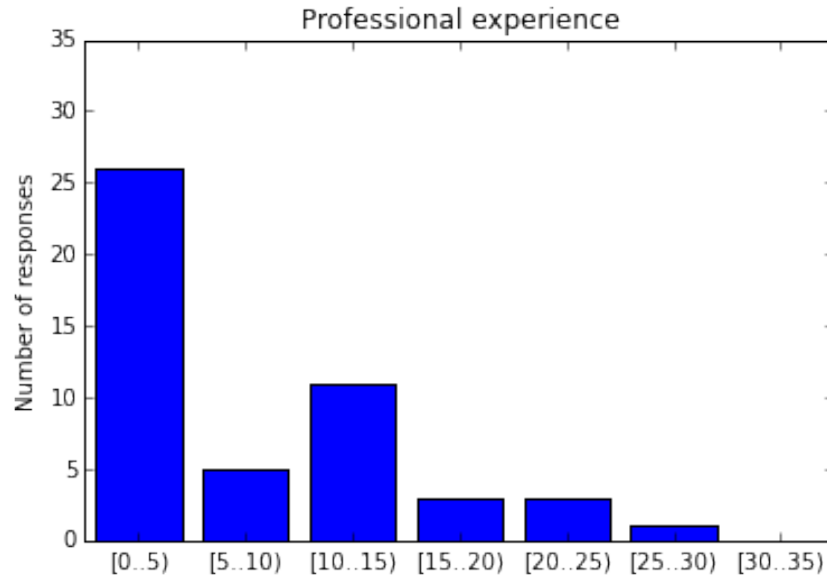
```
def iter_histogram(data, column_key):  
    v = [row[column_key] for row in data] # grab values  
    freqs, bins = np.histogram(v, bins=7, range=(0,35))  
    for i, freq in enumerate(freqs):  
        yield ('[{}..{}]'.format(int(bins[i]), int(bins[i+1])), freq)
```

```
def make_histogram_plot(data, column_key, title):  
    d = OrderedDict(iter_histogram(data, column_key))  
    keys = list(d.keys())  
    xs = [i+0.1 for i, _ in enumerate(keys)]  
    plt.bar(xs, d.values())  
    plt.ylabel('Number of responses')  
    plt.axis([0,7,0,35])  
    plt.title(title)  
    plt.xticks([i + 0.5 for i, _ in enumerate(keys)], keys)  
    plt.show()
```

Create plots for professional and programming experience

```
make_histogram_plot(data, BACKGROUND_YEARS_PROFESSIONAL, 'Professional experience')  
make_histogram_plot(data, BACKGROUND_YEARS_PROGRAMMING, 'Programming experience')
```

Histograms for experience (bin size 5 years)

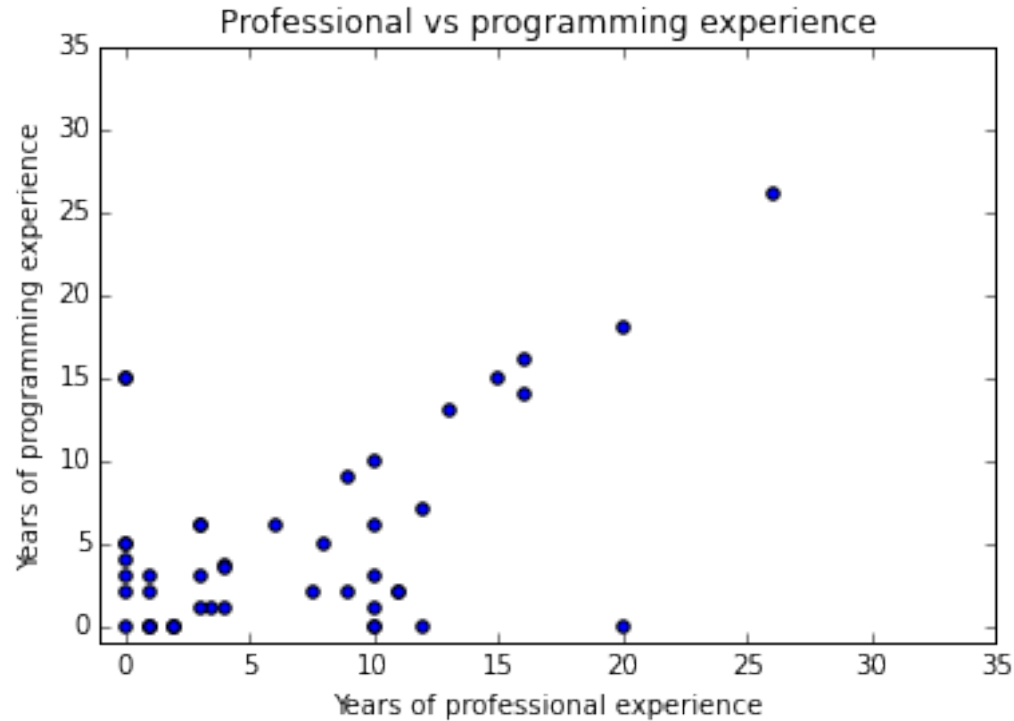


Creating a scatter plot

Create scatter plot

```
professional_experience = [row[BACKGROUND_YEARS_PROFESSIONAL] for row in data]
programming_experience = [row[BACKGROUND_YEARS_PROGRAMMING] for row in data]
plt.scatter(professional_experience, programming_experience)
plt.title('Professional vs programming experience')
plt.xlabel('Years of professional experience')
plt.ylabel('Years of programming experience')
plt.axis([-1,35,-1,35])
plt.show()
```


A scatter plot comparing experience



EXERCISE 4: Visualization with Matplotlib

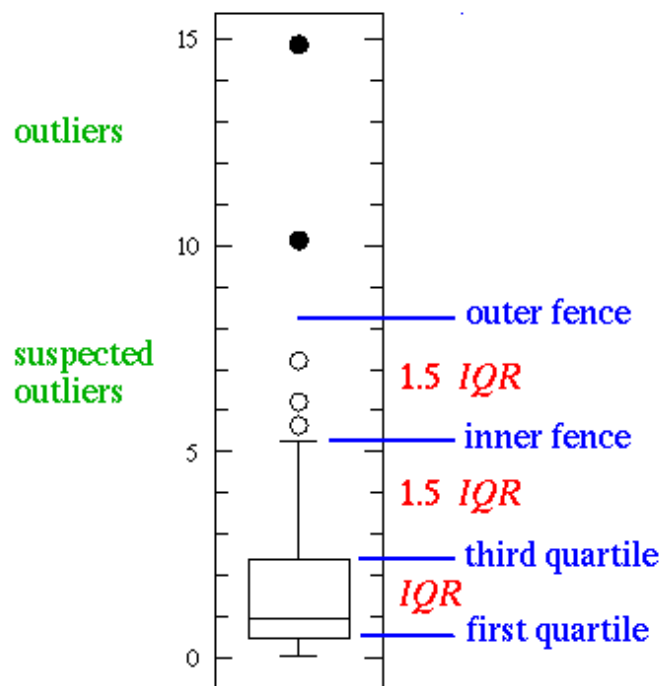
- Bar charts
 -  code cell under “Making a bar chart”
 - Complete code to create bar charts of know and future industries
- Histogram
 -  code cell under “Making a histogram”
 - Are mean and standard deviation useful summary statistics here?
- Scatter plots
 -  code cell under “Making a scatterplot”

Box plots and correlation

Using boxplots to compare distributions

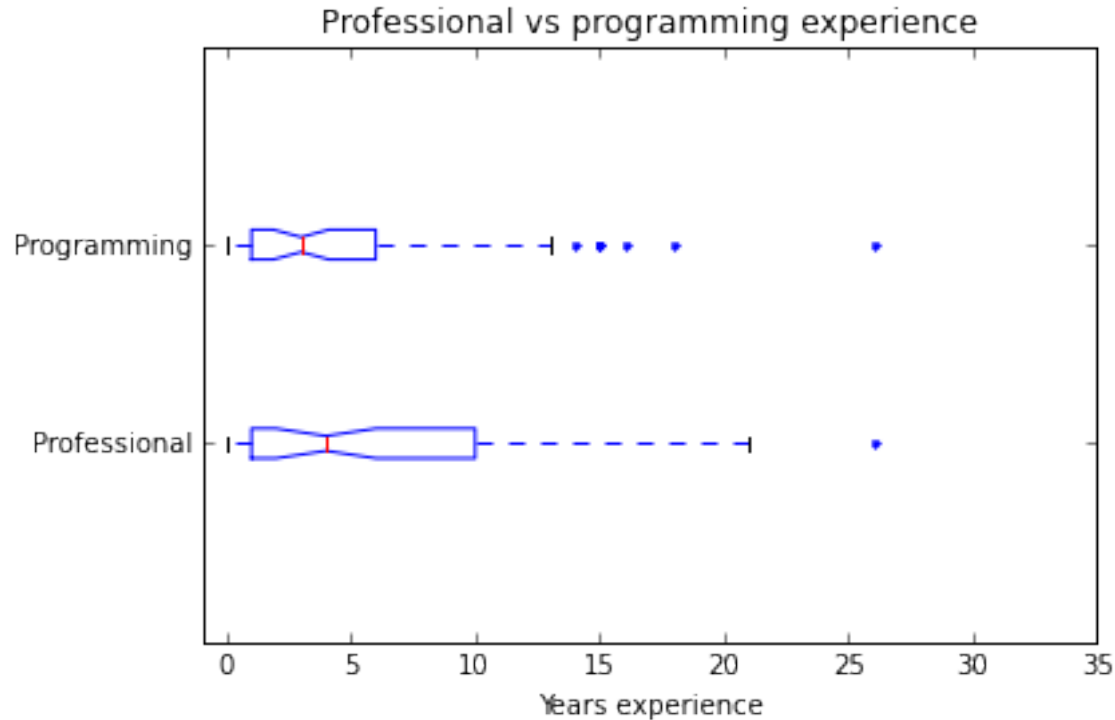
- Mean and stdev are not informative when data is skewed
- **Box plots** summarise data based on 5 numbers:
 - Lower inner fence – $Q1 - 1.5 * IQR$
 - First quartile ($Q1$) – equivalent to 25th percentile
 - Median ($Q2$) – equivalent to 50th percentile
 - Third quartile ($Q3$) – equivalent to 75th percentile
 - Upper inner fence – $Q3 + 1.5 * IQR$
- Values outside fences are outliers
- Sometimes include outer fences at $3 * IQR$

Box Plots illustrated





A box plot comparing experience distributions



Using correlation statistics to measure dependence

- Scipy includes various correlation statistics
 - Pearson's r for two normally distributed variables
 - Spearman's ρ for ratio data, ordinal data, etc (rank-order correlation)
 - Kendall's τ for ordinal variables
- List of various scipy statistics including correlation coefficients:
<http://docs.scipy.org/doc/scipy-0.14.0/reference/stats.html>



Calculating correlation

Since correlation is paired,
grab values where both
variables are defined

```
from scipy import stats
# only keep rows where both professional and programming experience are defined
prof, prog = [], []
for row in data:
    if row[BACKGROUND_YEARS_PROFESSIONAL] is np.nan:
        continue # ignore rows with no value for professional experience
    elif row[BACKGROUND_YEARS_PROGRAMMING] is np.nan:
        continue # ignore rows with no value for programming experience
    else:
        prof.append(row[BACKGROUND_YEARS_PROFESSIONAL])
        prog.append(row[BACKGROUND_YEARS_PROGRAMMING])
print("Pearson (r, p): {}".format(stats.pearsonr(prof, prog)))
print(stats.spearmanr(prof, prog))
```

Calculate Person's r
and Spearman's rho

EXERCISE 5: Box plots and correlation

- Box plots
 -  code cell under “Visualising distributions with box plots”
 - Which experience variable has more outliers?
 - Which experience variable has a tighter distribution?
- Correlation
 -  code cell under “Calculating correlation between two variables”

Review



Levels of Measurement

	Nominal	Ordinal	Interval	Ratio
Countable	✓	✓	✓	✓
Order defined		✓	✓	✓
Difference defined (addition, subtraction)			✓	✓
Zero defined (multiplication, division)				✓



Measures of Central Tendency

	Nominal	Ordinal	Interval	Ratio
Mode	✓	✓	✓	✓
Median		✓	✓	✓
Mean			✓	✓



Measures of Dispersion

	Nominal	Ordinal	Interval	Ratio
Counts / Distribution	✓	✓	✓	✓
Minimum, Maximum		✓	✓	✓
Range		✓	✓	✓
Percentiles		✓	✓	✓
Standard deviation, Variance			✓	✓

Notes

- Python a good example of a scripting language for DS
- programmatic approaches allow for more powerful / flexible data preparation and analysis,
 - and more control on the visualisations
- Many useful support libraries available in the Python ecosystem
 - numpy, scipy, matplotlib

Additional reading (not examinable)

- matplotlib API reference
https://matplotlib.org/2.0.2/api/pyplot_api.html
- NumPy and SciPy documentation
<http://docs.scipy.org/doc/>
- Data Analysis with Python (O'Reilly)
<http://shop.oreilly.com/product/0636920023784.do>

Participation

Requirements

- Submit code at end of each week
- **Jupyter Notebooks:**
 - The various exercises have placeholder cells marked as TODO:

```
# TODO: replace the content of this cell  
raise NotImplementedError
```

- The content of these cells needs to be replaced with your own solution
=> basis for participation marking

Output

- Code/spreadsheets from exercises
- Each week's participation assessed as:
all done, partially done, no participation

Marking

- 10% of overall mark

TODO until Monday next week:

***Export your Jupyter notebooks as HTML and
upload to Canvas***

Next Time

Next Lecture : Hypothesis Testing and Evaluation

Objective

Learn Python tools for exploring a new data set programmatically.

Lecture

- Questions: Do two populations differ?
Which approach is better?
- Significance: pairwise t-tests, confidence intervals, non-normal data
- Evaluation: cluster evaluation, classifier evaluation, user evaluation

Readings

- Model evaluation (sklearn doco)
http://scikit-learn.org/stable/modules/model_evaluation.html#model-evaluation
- Hypothesis testing (scipy lectures)
<http://www.scipy-lectures.org/packages/statistics/index.html#hypothesis-testing-comparing-two-groups>

Exercises

- scipy: statistical tests
- sklearn: evaluation metrics

TODO in W3

- Learning Python
- Summarise and prepare data

Questions?